



AIN SHAMS UNIVERSITY
FACULTY OF ENGINEERING

Job Market Intelligent System

BigData Project (CSE476s)

Submitted to: Dr/Mohamed Sobh

Team Members

1. Mostafa Al Hassan Salah – 2200747
2. Mosa Abdulaziz Morga Abdulaziz – 2200257
3. Mohamed Khaled Ahmed – 2201057
4. Ibrahim Mahmoud Ibrahim – 2200182
5. Youssef Medhat Mahmoud – 2200626
6. Ahmed Naggar Hassan – 2200306
7. Mario Milad Helmy – 2200540
8. Omar Ahmed Mohamed – 2200267
9. Ahmed Sayed Abdullah – 2200737

Contents

1	Introduction	3
1.1	Problem Statement	3
1.2	Business and Market Benefits	3
2	System Architecture	3
2.1	Architecture Overview Diagram	4
2.2	Request Flow Sequence	5
3	Database and Schema	5
3.1	ER-style Collection Diagram	6
3.2	Enum Types	6
4	Services	6
4.1	Wuzzuf Scraper Service (<code>wuzzuf_scraper.py</code>)	6
4.2	AI Enrichment Service (<code>Ai_Enrich_Service.py</code>)	6
4.3	Study Plan Service (<code>studyplan_service.py</code>)	7
5	Controllers	7
5.1	Scraper Controller	7
5.2	Analytics Controller	7
5.3	Jobs Controller	7
5.4	Study Plan Controller	7
5.5	Controller Interaction Diagram	8
6	API Documentation	8
6.1	Scraping Endpoints	8
6.2	Jobs Endpoints	8
6.3	Insights Endpoints	9
6.4	Study Plan Endpoint	9
7	Frontend Hierarchy	9
7.1	Main Structure	10
7.2	Component Tree Diagram	10
7.3	Frontend Data Flow Diagram	10
8	Future Improvements	11
9	Technologies Used	12
10	Conclusion	13

1 Introduction

The Job Market Intelligent System is an end-to-end platform that collects job postings, enriches them with AI, and presents actionable analytics to job seekers and decision makers. The system combines a Python/FastAPI backend, MongoDB storage, and a React frontend to transform raw job-board content into structured market intelligence.

1.1 Problem Statement

The Egyptian job market has fragmented and noisy job data across multiple sources, making it difficult for users to answer practical questions quickly. Candidates often need to manually inspect many listings to understand:

- which skills are currently in demand,
- what seniority level a role maps to,
- and what salary range to expect when salary is missing.

Without a centralized and enriched view, decision quality is reduced for both job seekers and organizations monitoring hiring trends.

1.2 Business and Market Benefits

The platform delivers clear market value through:

- centralized analytics dashboards for category, seniority, and trend monitoring,
- AI-powered extraction of normalized skills, role category, and seniority,
- salary estimation when postings omit compensation details,
- interactive exploration tools (filters, comparisons, graphs),
- AI study-plan assistance to support candidate upskilling,
- clustering-based market intelligence that reveals skill co-occurrence patterns and company skill demand.

2 System Architecture

The system follows a layered architecture with clear responsibility boundaries:

- **Client Layer:** React single-page application for browsing jobs and analytics.
- **API Layer:** FastAPI routers expose endpoints for jobs, insights (including clustering), scraping, and study plan chat.
- **Controller + Service Layer:** controllers orchestrate workflows; services implement scraping, AI enrichment, and response generation.
- **Data Layer:** MongoDB stores job and crawl log collections.

- **External Integrations:** Wuzzuf pages for source data, Google Gemini/Ollama for AI enrichment and study-plan generation.

The clustering feature derives a skill co-occurrence matrix where each cell counts how often two skills appear together across postings:

$$\text{co_occurrence}(s_i, s_j) = \sum_{job \in J} \mathbf{1}[s_i \in job \wedge s_j \in job]$$

This matrix is then used to detect strongly related skill groups (clusters) and render heatmap-based insights on the frontend.

2.1 Architecture Overview Diagram

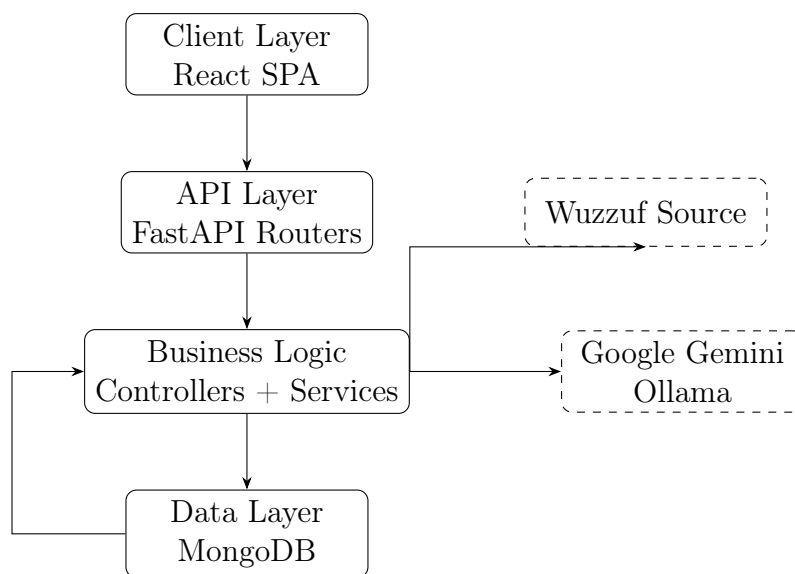


Figure 1: High-level architecture of the Job Market Intelligent System.

2.2 Request Flow Sequence

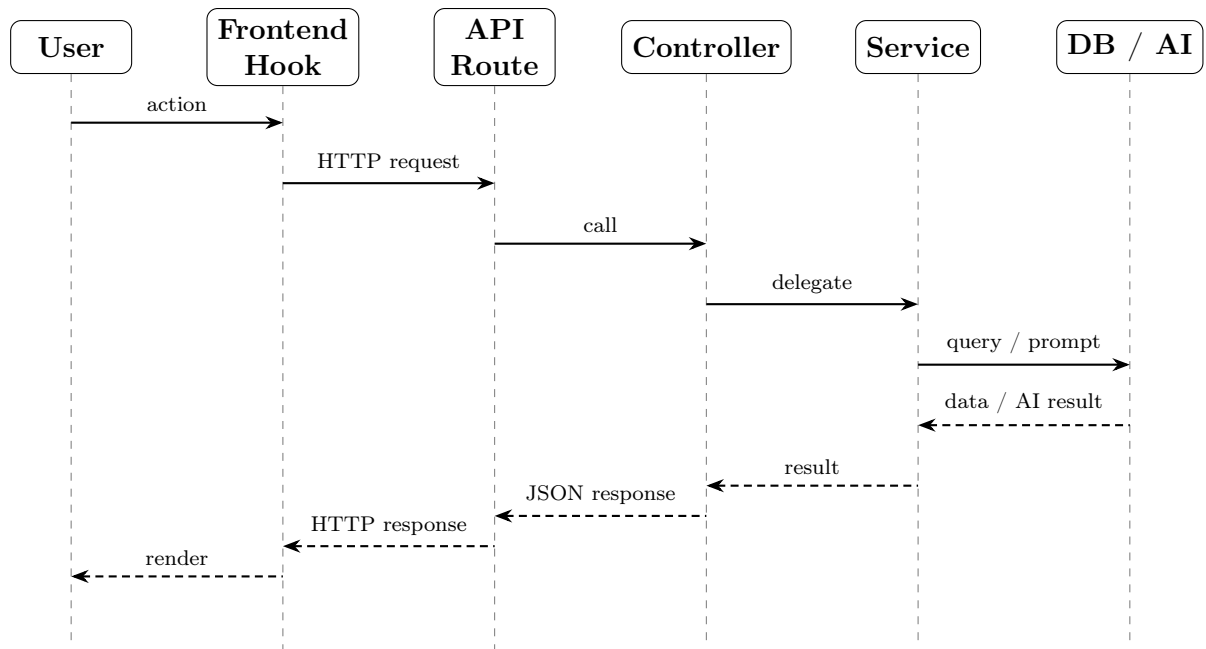


Figure 2: Request flow sequence for analytics, jobs, and AI-powered endpoints.

3 Database and Schema

The backend uses MongoDB with two main collections:

- **jobs**: stores scraped jobs and enrichment outputs.
- **crawl_logs**: stores scrape execution summaries and status.

Indexes are created at startup in `src/models/database.py`. For jobs, indexes include `source_url` (unique), `company+title`, `posted_date`, `location`, `listed_skills`, `normalized_skills`, `category+seniority`, and `scraped_at`.

3.1 ER-style Collection Diagram

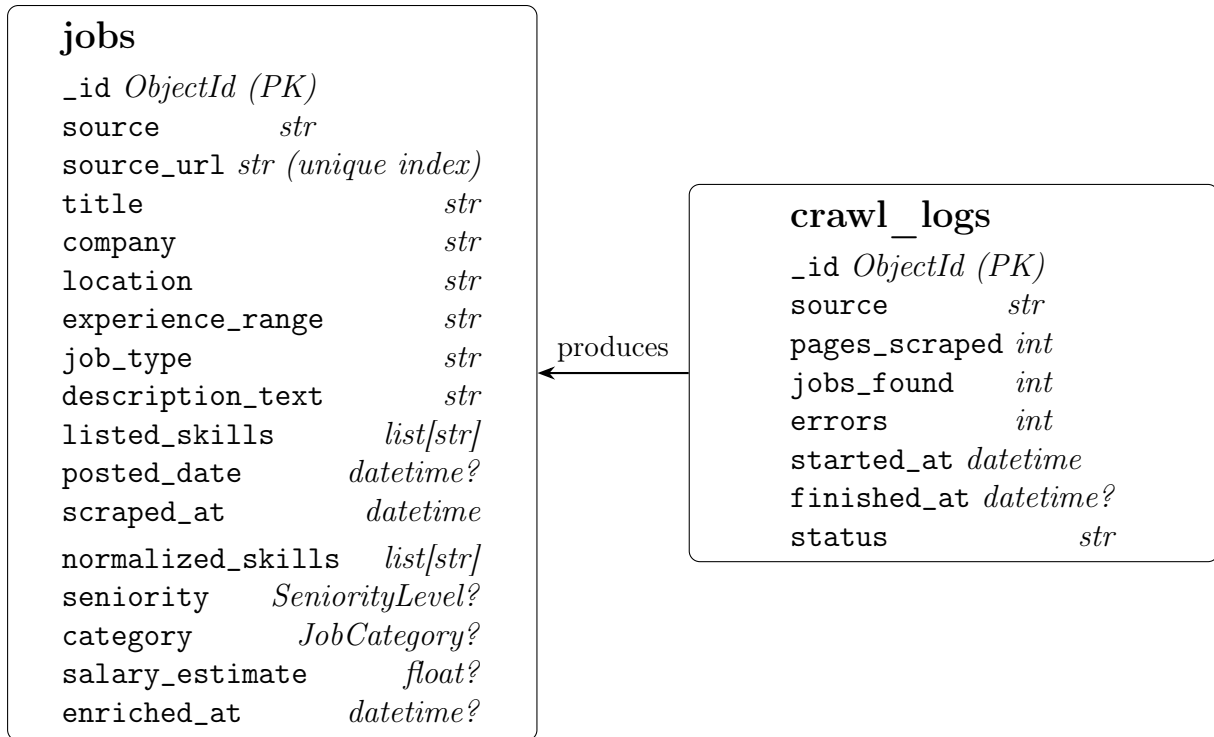


Figure 3: MongoDB collections with field schemas and their logical relationship.

3.2 Enum Types

- **JobCategory:** backend, frontend, ai, fullstack, data, devops, mobile, design, management, qa, other.
- **SeniorityLevel:** junior, mid, senior, lead.

4 Services

4.1 Wuzzuf Scraper Service (`wuzzuf_scraper.py`)

This service is responsible for fetching job postings from Wuzzuf. It supports:

- multi-keyword search with a broad default technical keyword set,
- configurable page depth and delays,
- progressive callbacks for scrape status updates,
- detailed parsing and normalization into `JobCreate` objects.

4.2 AI Enrichment Service (`Ai_Enrich_Service.py`)

Despite its filename, the implementation is now Gemini-first. It provides:

- extraction of skills, seniority, category, and salary hints from job descriptions,
- Gemini API integration with retry and backoff for quota/rate-limit resilience,
- fallback to Ollama when cloud AI is unavailable,
- deterministic rule-based fallback when AI responses are unavailable or malformed.

4.3 Study Plan Service (`studyplan_service.py`)

This service powers chat-style guidance for learning paths:

- accepts a user message describing learning goals,
- generates structured study guidance via Gemini (or Ollama fallback),
- returns assistant-style content consumed by the frontend chat widget.

5 Controllers

Controllers coordinate request workflows between API routes, services, and MongoDB.

5.1 Scraper Controller

`enqueue_scrape()` registers a task and returns a task ID; `run_scrape()` performs scraping, applies AI enrichment with bounded concurrency, inserts jobs, and updates crawl logs.

5.2 Analytics Controller

`get_dashboard()`, `get_skill_graph()`, and `get_salary_intelligence()` aggregate data from MongoDB to produce charts, KPIs, percentiles, trends, and comparisons. Clustering methods (`get_skill_clustering()`, `get_company_hiring_patterns()`, `get_company_skill_m`, `get_category_hiring_trends()`) provide co-occurrence heatmaps, company-level skill-demand summaries, and category trend analytics.

5.3 Jobs Controller

`list_jobs()` applies pagination and filters; `get_job_by_id()` fetches one job and serializes it for API output.

5.4 Study Plan Controller

`chat()` validates the request and delegates response generation to the study plan service.

5.5 Controller Interaction Diagram

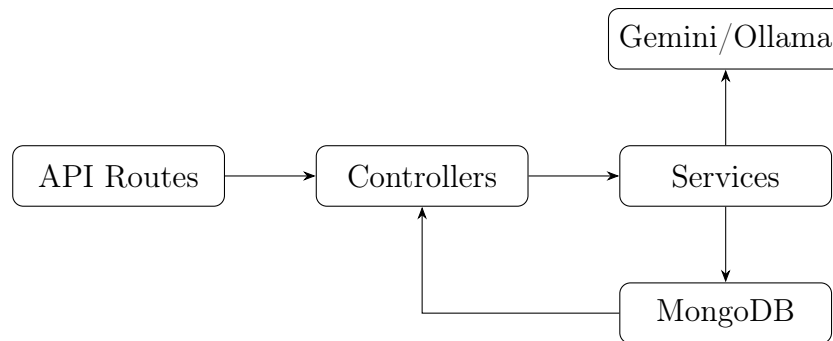


Figure 4: Typical controller orchestration path.

6 API Documentation

6.1 Scraping Endpoints

Method	Path	Role	Request/Response
POST	/scrape/jobs	Trigger async scrape task	Body: keywords[], max_pages; Response: task_id, status
GET	/scrape/status/{task_id}	Poll scrape progress	Response: status fields (pages, jobs, errors, progress)
POST	/scrape/clear?confirm=true	Clear jobs and crawl logs	Response: deleted counts

Table 1: Scraper API routes.

6.2 Jobs Endpoints

Method	Path	Role	Request/Response
GET	/api/jobs	Paginated and filtered jobs list	Query: page, per_page, company, location, skill, seniority, category, search, sort_by
GET	/api/jobs/{job_id}	Retrieve one job details record	Response: JobResponse object

Table 2: Jobs API routes.

6.3 Insights Endpoints

Method	Path	Role	Request/Response
GET	/api/insights/dashboard	KPI cards and dashboard aggregations	Query: category, seniority, date_from, date_to
GET	/api/insights/skill-graph	Skill co-occurrence graph	Query: min_weight; Response: nodes + edges
GET	/api/insights/salary-intelligence	Salary percentiles/distribution by company, location	Query: category, seniority, location
GET	/api/insights/skill-clustering	Co-occurrence heatmap and skill clusters	Query: min_skill_frequency, category, seniority
GET	/api/insights/company-hiring-patterns	Company hiring profile summaries	Response: companies, top skills, category/seniority distribution
GET	/api/insights/company-skill-matrix	Company-vs-skills heatmap data	Response: companies, skills, heatmap
GET	/api/insights/category-hiring-trends	Hiring trend summary by category	Response: totals, avg salary, seniority breakdown

Table 3: Insights API routes.

6.4 Study Plan Endpoint

Method	Path	Role	Request/Response
POST	/api/studyplan/chat	Generate AI learning guidance	Body: message; Response: role + content

Table 4: Study plan API route.

7 Frontend Hierarchy

The frontend is built with React 18 + TypeScript and organized into pages, reusable components, hooks, stores, and API helpers. Global providers include React Query and BrowserRouter.

7.1 Main Structure

- **Pages:** Landing, Job Browser, Job Detail, Dashboard, Compare, Salary Statistics, Clustering Analysis.
- **Components:** navigation, cards, filter controls, pagination, chart components, and study-plan chat widget.
- **Hooks:** useJobs, useInsights, useScrape, useStudyPlan, useClustering.
- **State Stores:** Zustand-based filterStore and themeStore.
- **Data Layer:** Axios client + React Query caching.

7.2 Component Tree Diagram

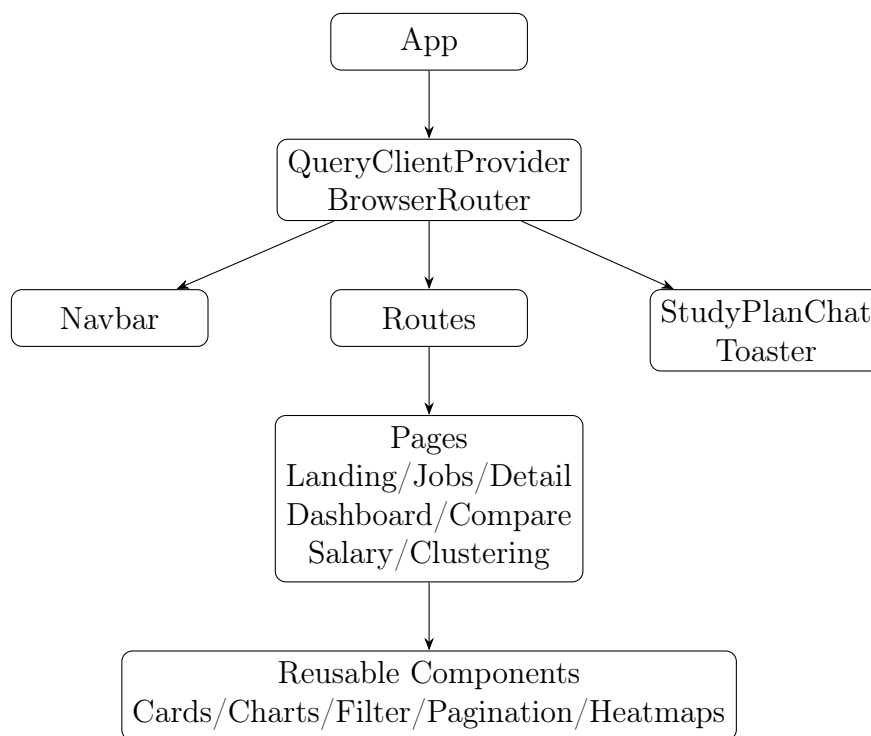


Figure 5: Frontend component hierarchy overview.

7.3 Frontend Data Flow Diagram

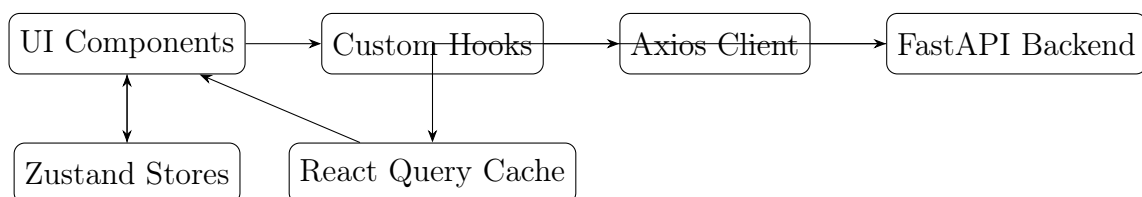


Figure 6: Frontend state and data movement.

8 Future Improvements

One impactful next step is introducing a Retrieval-Augmented Generation (RAG) pipeline for stronger AI quality and grounding:

- index curated market documents and historical job data in a vector store,
- retrieve the most relevant chunks per user query or enrichment request,
- pass retrieved context with prompts to the LLM before generation,
- log retrieval quality and model output quality for iterative tuning.

Expected benefits include fewer hallucinations, higher factual consistency, and more context-aware recommendations.

9 Technologies Used

Technology	Category	Example Usage in Project
Python	Backend Language	Core backend logic, controllers, services, and data models
FastAPI	Web Framework	REST API routing and OpenAPI docs
MongoDB	Database	Storing jobs and crawl logs
Motor/PyMongo	DB Driver	Async Mongo operations and index management
Pydantic	Validation	Request/response and internal schema models
Google Gemini (google-genai)	AI Service	Skill extraction, category/seniority inference, study-plan responses
Ollama	Local AI Fallback	Backup generation when cloud AI is unavailable
Scrapling + BeautifulSoup + lxml	Scraping/Parsing	Wuzzuf page crawling and field extraction
React + TypeScript	Frontend	SPA pages and typed UI logic
Vite	Frontend Build Tool	Development server and optimized production builds
Tailwind CSS	Styling	Utility-first UI design system
Zustand	Client State	Theme and filter store management
React Query	Server State	API fetching, caching, and invalidation
Recharts + D3	Visualization	Charts, trend lines, and skill graph rendering
Docker + Compose	Deployment/DevOps	Containerized backend/frontend/database services
Git	Version Control	Branching, commits, and release workflow

Table 5: Technology stack summary.

10 Conclusion

The Job Market Intelligent System demonstrates a practical architecture for converting raw job postings into usable intelligence. By integrating scraping, AI enrichment, robust backend APIs, clustering-based analytics (skill co-occurrence and company demand), and a modern frontend, the platform supports both exploration and decision-making for users tracking the Egyptian technology job market.

The design is extensible: future enhancements such as RAG-powered grounding, additional data sources, and deeper salary modeling can be integrated without restructuring the whole system. This makes the project suitable as both an academic deliverable and a scalable foundation for real-world market intelligence products.